

The Multi-Agent Trap

Kaushik Rajan

has handled [2.3 million customer conversations in a single month](#). That's the workload of 700 full-time human agents. Resolution time dropped from 11 minutes to under 2. Repeat inquiries fell 25%. Customer satisfaction scores climbed 47%. Cost per service transaction: \$0.32 down to \$0.19. Total savings through late 2025: roughly [\\$60 million](#).

The system runs on a multi-agent architecture built with LangGraph.

Here's the other side. [Gartner predicted that over 40% of agentic AI projects will be canceled](#) by the end of 2027. Not scaled back. Not paused. Canceled. Escalating costs, unclear business value, and inadequate risk controls.

Same technology. Same year. Wildly different outcomes.

If you're building a multi-agent system (or evaluating whether you should), the gap between these two stories contains everything you need to know. This playbook covers three architecture patterns that work in production, the five failure modes that kill projects, and a framework comparison to help you choose the right tool. You'll walk away with a pattern selection guide and a pre-deployment checklist you can use on Monday morning.

Why More AI Agents Usually Makes Things Worse

The intuition feels solid. Split complex tasks across specialized agents, let each one handle what it's best at. Divide and conquer.

In December 2025, a Google DeepMind team led by Yubin Kim [tested this assumption rigorously](#). They ran 180 configurations across 5 agent architectures and 3 Large Language Model (LLM) families. The finding should be taped above every AI team's monitor:

Unstructured multi-agent networks amplify errors up to 17.2 times compared to single-agent baselines.

Not 17% worse. Seventeen times worse.

When agents are thrown together without structured topology (what the paper calls a "bag of agents"), each agent's output becomes the next agent's input. Errors don't cancel. They cascade.

Picture a pipeline where Agent 1 extracts customer intent from a support ticket. It misreads "billing dispute" as "billing inquiry" (subtle, right?). Agent 2 pulls the wrong response template. Agent 3 generates a reply that addresses the wrong problem entirely. Agent 4 sends it. The customer responds, angrier now. The system processes the angry reply through the same broken chain. Each loop amplifies the original misinterpretation. That's the 17x effect in practice: not a catastrophic failure, but a quiet compounding of small errors that produces confident nonsense.

The same study found a saturation threshold: coordination gains plateau beyond 4 agents. Below that number, adding agents to a structured system helps. Above it, coordination overhead consumes the benefits.

This isn't an isolated finding. The [Multi-Agent Systems Failure Taxonomy \(MAST\)](#) study, published in March 2025, analyzed 1,642 execution traces across 7 open-source frameworks. Failure rates ranged from 41% to 86.7%. The largest failure category: coordination breakdowns at 36.9% of all failures.

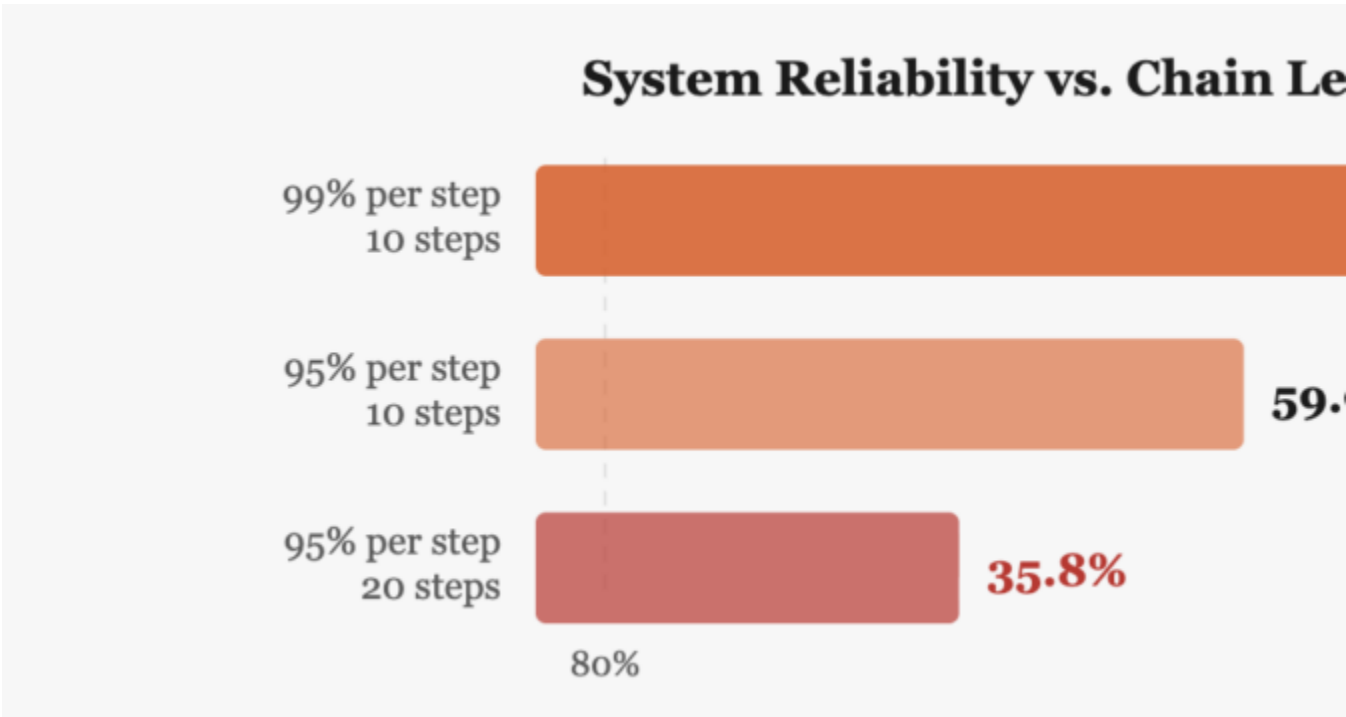
The obvious counter-argument: these failure rates reflect immature tooling, not a fundamental architecture problem. As models improve, the compound reliability issue shrinks. There's truth in this. Between January 2025 and January 2026, single-agent task completion rates improved significantly ([Carnegie Mellon benchmarks](#) showed the best agents reaching 24% on complex office tasks, up from near-zero). But even at 99% per-step reliability, the compound math still applies. Better models shift the curve. They don't eliminate the compound effect. Architecture still determines whether you land in the 60% or the 40%.

The Compound Reliability Problem

Here's the arithmetic that most architecture documents skip.

A single agent completes a step with 99% reliability. Sounds excellent. Chain 10 sequential steps: $0.99^{10} = 90.4\%$ overall reliability.

Drop to 95% per step (still strong for most AI tasks). Ten steps: $0.95^{10} = 59.9\%$. Twenty steps: $0.95^{20} = 35.8\%$.



Compound reliability decay: agents that succeed individually produce systems that fail collectively. Image by the author.

You started with agents that succeed 19 out of 20 times. You ended with a system that fails nearly two-thirds of the time.

Token costs compound too. A document analysis workflow consuming 10,000 tokens with a single agent

requires 35,000 tokens across a 4-agent implementation. That's a 3.5x cost multiplier before you account for retries, error handling, and coordination messages.

This is why Klarna's architecture works and most copies of it don't. The difference isn't agent count. It's topology.

Three Multi-Agent Patterns That Work in Production

Flip the question. Instead of asking "how many agents do I need?", ask: "how would I definitely fail at multi-agent AI?" The research answers clearly. By chaining agents without structure. By ignoring coordination overhead. By treating every problem as a multi-agent problem when a single well-prompted agent would suffice.

Three patterns avoid these failure modes. Each serves a different task shape.

Plan-and-Execute

A capable model creates the complete plan. Cheaper, faster models execute each step. The planner handles reasoning; the executors handle doing.

This is close to what Klarna runs. A frontier model analyzes the customer's intent and maps resolution steps. Smaller models execute each step: pulling account data, processing refunds, generating responses. The planning model touches the task once. Execution models handle the volume.

The cost impact: routing planning to one capable model and execution to cheaper models [cuts costs by up to 90%](#) compared to using frontier models for everything.

When it works: Tasks with clear goals that decompose into sequential steps. Document processing, customer service workflows, research pipelines.

When it breaks: Environments that change mid-execution. If the original plan becomes invalid halfway through, you need re-planning checkpoints or a different pattern entirely. This is a one-way door if your task environment is volatile.

Supervisor-Worker

A supervisor agent manages routing and decisions. Worker agents handle specialized subtasks. The supervisor breaks down requests, delegates, monitors progress, and consolidates outputs.

Google DeepMind's research validates this directly. A centralized control plane suppresses the 17x error amplification that "bag of agents" networks produce. The supervisor acts as a single coordination point, preventing the failure mode where (for example) a support agent approves a refund while a compliance agent simultaneously blocks it.

When it works: Heterogeneous tasks requiring different specializations. Customer support with escalation paths, content pipelines with review stages, financial analysis combining multiple data sources.

When it breaks: When the supervisor becomes a bottleneck. If every decision routes through one agent, you've recreated the monolith you were trying to escape. The fix: give workers bounded autonomy on decisions within their domain, escalate only edge cases.

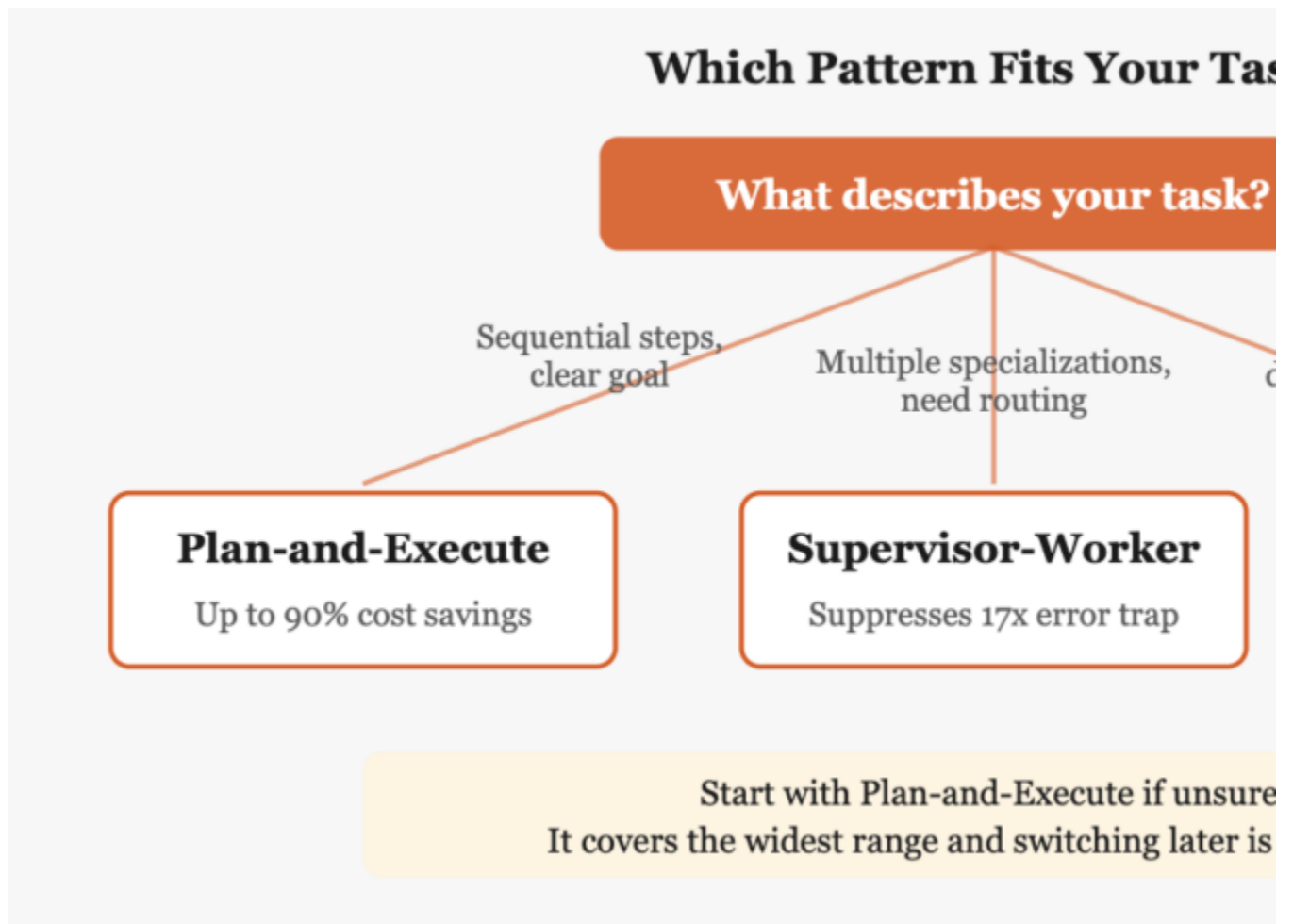
Swarm (Decentralized Handoffs)

No supervisor. Agents hand off to each other based on context. Agent A handles intake, determines this is a billing issue, and passes to Agent B (billing specialist). Agent B resolves it or passes to Agent C (escalation) if needed.

OpenAI's original Swarm framework was educational only (they said so explicitly in the README). Their production-ready Agents Software Development Kit (SDK), released in March 2025, implements this pattern with guardrails: each agent declares its handoff targets, and the framework enforces that handoffs follow declared paths.

When it works: High-volume, well-defined workflows where routing logic is embedded in the task itself. Chat-based customer support, multi-step onboarding, triage systems.

When it breaks: Complex handoff graphs. Without a supervisor, debugging “why did the user end up at Agent F instead of Agent D?” requires production-grade observability tools. If you don't have distributed tracing, don't use this pattern.



Pattern selection decision tree. When in doubt, start simple and graduate up. Image by the author.

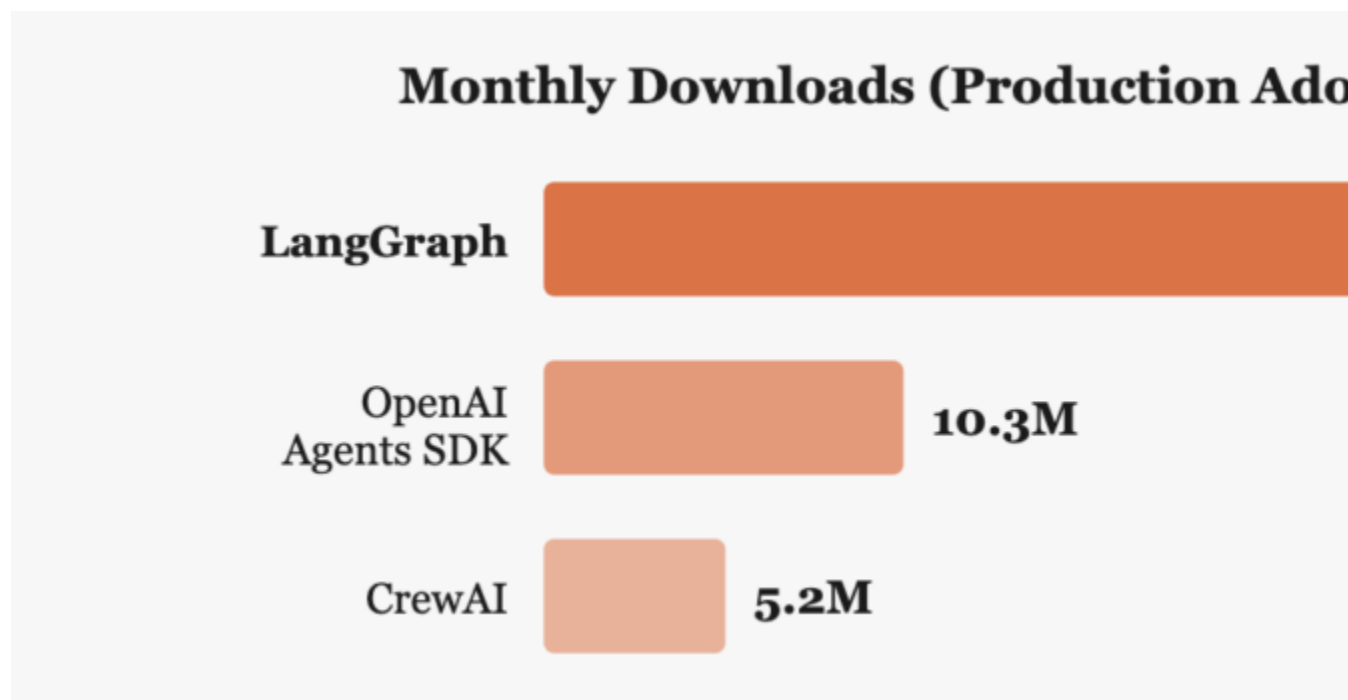
Which Multi-Agent Framework to Use

Three frameworks dominate production multi-agent deployments right now. Each reflects a different philosophy about how agents should be organized.

LangGraph uses graph-based state machines. [34.5 million monthly downloads](#). Typed state schemas enable precise checkpointing and inspection. This is what Klarna runs in production. Best for stateful workflows where you need human-in-the-loop intervention, branching logic, and durable execution. The trade-off: steeper learning curve than alternatives.

CrewAI organizes agents as role-based teams. 44,300 GitHub stars and growing. Lowest barrier to entry: define agent roles, assign tasks, and the framework handles coordination. Deploys teams roughly [40% faster than LangGraph](#) for straightforward use cases. The trade-off: limited support for cycles and complex state management.

OpenAI Agents SDK provides lightweight primitives (Agents, Handoffs, Guardrails). The only major framework with equal Python and TypeScript/JavaScript support. Clean abstraction for the Swarm pattern. The trade-off: tighter coupling to OpenAI's models.



Downloads don't tell the whole story (CrewAI has more GitHub stars), but they're the best proxy for production adoption. Image by the author.

One protocol worth knowing: [Model Context Protocol \(MCP\)](#) has become the de facto interoperability standard for agent tooling. Anthropic donated it to the Linux Foundation in December 2025 (co-founded by Anthropic, Block, and OpenAI under the Agentic AI Foundation). Over 10,000 active public MCP servers exist. All three frameworks above support it. If you're evaluating tools, MCP compatibility is table stakes.

A starting point: If you're unsure, start with Plan-and-Execute on LangGraph. It's the most battle-tested combination. It handles the widest range of use cases. And switching patterns later is a reversible decision (a two-way door, in decision theory terms). Don't over-architect on day one.

Five Ways Multi-Agent Systems Fail

The MAST study identified 14 failure modes across 3 categories. The five below account for the majority of production failures. Each includes a specific prevention measure you can implement before your next deployment.

Pre-Deployment Checklist: The Five Failure Modes

1. Compound Reliability Decay

Calculate your end-to-end reliability before you ship. Multiply per-step success rates across your full chain. If the number drops below 80%, reduce the chain length or add verification checkpoints.

Prevention: Keep chains under 5 sequential steps. Insert a verification agent at step 3 and step 5 that checks output quality before passing downstream. If verification fails, route to a human or a fallback path (not a retry of the same chain).

2. Coordination Tax (36.9% of all MAS failures)

When two agents receive ambiguous instructions, they interpret them differently. A support agent approves a refund; a compliance agent blocks it. The user receives contradictory signals.

Prevention: Explicit input/output contracts between every agent pair. Define the data schema at every boundary and validate it. No implicit shared state. If Agent A's output feeds Agent B, both agents must agree on the format before deployment, not at runtime.

3. Cost Explosion

Token costs multiply across agents (3.5x in documented cases). Retry loops can burn through \$40 or more in Application Programming Interface (API) fees within minutes, with no useful output to show for it.

Prevention: Set hard per-agent and per-workflow token budgets. Implement circuit breakers: if an agent exceeds its budget, halt the workflow and surface an error rather than retrying. Log cost per completed workflow to catch regressions early.

4. Security Gaps

The Open Worldwide Application Security Project (OWASP) Top 10 for LLM Applications found prompt injection vulnerabilities in [73% of assessed production deployments](#). In multi-agent systems, a compromised agent can propagate malicious instructions to every downstream agent.

Prevention: Input sanitization at every agent boundary, not just the entry point. Treat inter-agent messages with the same suspicion you'd apply to external user input. Run a red-team exercise against your agent chain before production launch.

5. Infinite Retry Loops

Agent A fails. It retries. Fails again. In multi-agent systems, Agent A's failure triggers Agent B's error handler, which calls Agent A again. The loop runs until your budget runs out.

Prevention: Maximum 3 retries per agent per workflow execution. Exponential backoff between retries. Dead-letter queues for tasks that fail past the retry limit. And one absolute rule: never let one agent trigger another without a cycle check in the orchestration layer.

Prompt injection was found in 73% of production LLM deployments assessed during security audits. In multi-agent systems, one compromised agent can propagate the attack downstream.

Tool vs. Worker: The \$60 Million Architecture Gap

In February 2026, the [National Bureau of Economic Research \(NBER\) published a study](#) surveying nearly 6,000 executives across the US, UK, Germany, and Australia. The finding: 89% of firms reported zero change in productivity from AI. Ninety percent of managers said AI had no impact on employment. These firms averaged 1.5 hours per week of AI use per executive.

[Fortune called it](#) a resurrection of Robert Solow's 1987 paradox: "You can see the computer age everywhere but in the productivity statistics." History is repeating, forty years later, with a different technology and the same pattern.

The 90% seeing zero impact deployed AI as a tool. The companies saving millions deployed AI as workers.

The contrast with Klarna isn't about better models or bigger compute budgets. It's a structural choice. The 90% treated AI as a copilot: a tool that assists a human in a loop, used 1.5 hours per week. The companies seeing real returns (Klarna, Ramp, Reddit via Salesforce Agentforce) treated AI as a workforce: autonomous agents executing structured workflows with human oversight at decision boundaries, not at every step.

That's not a technology gap. It's an architecture gap. The opportunity cost is staggering: the same engineering budget producing zero Return on Investment (ROI) versus \$60 million in savings. The variable isn't spend. It's structure.

Forty percent of agentic AI projects will be canceled by 2027. The other sixty percent will ship. The difference won't be which LLM they chose or how much they spent on compute. It will be whether they understood three patterns, ran the compound reliability math, and built their system to survive the five failure modes that kill everything else.

Klarna didn't deploy 700 agents to replace 700 humans. They built a structured multi-agent system where a smart planner routes work to cheap executors, where every handoff has an explicit contract, and where the architecture was designed to fail gracefully rather than cascade.

You have the same patterns, the same frameworks, and the same failure data. The playbook is open. What you build with it is the only remaining variable.

References

1. Kim, Y. et al. [“Towards a Science of Scaling Agent Systems.”](#) Google DeepMind, December 2025.
2. Cemri, M., Pan, M.Z., Yang, S. et al. [“MAST: Multi-Agent Systems Failure Taxonomy.”](#) March 2025.
3. Coshow, T. and Zamanian, K. [“Multiagent Systems in Enterprise AI.”](#) Gartner, December 2025.
4. Gartner. [“Over 40 Percent of Agentic AI Projects Will Be Canceled by End of 2027.”](#) June 2025.
5. LangChain. [“Klarna: AI-Powered Customer Service at Scale.”](#) 2025.
6. Klarna. [“AI Assistant Handles Two-Thirds of Customer Service Chats in Its First Month.”](#) 2024.
7. Bloom, N. et al. [“Firm Data on AI.”](#) National Bureau of Economic Research, Working Paper #34836, February 2026.
8. Fortune. [“Thousands of CEOs Just Admitted AI Had No Impact on Employment or Productivity.”](#) February 2026.
9. Moran, S. [“Why Your Multi-Agent System Is Failing: Escaping the 17x Error Trap.”](#) Towards Data Science, January 2026.
10. Carnegie Mellon University. [“AI Agents Fail at Office Tasks.”](#) 2025.
11. Redis. [“AI Agent Architecture: Patterns and Best Practices.”](#) 2025.
12. DataCamp. [“CrewAI vs LangGraph vs AutoGen: Comparison Guide.”](#) 2025.